Beyond STRIDE

Securing Agentic AI with the MAESTRO Framework

Darylynn Ross

Sr. Application Security Engineer, Product Security
CoverMyMeds, part of McKesson Corporation

Disclaimer: I do a ton of threat modeling, BUT I am not an expert on MAESTRO yet. I am presenting to you because I am learning this myself and want to share it with you.

Please don’t hold questions to the end.

Discussion is encouraged.

Agenda

Topics for Today

1. **EchoLeak Case Study**
2. **Threat Modeling Basics and STRIDE**
3. **How Agentic AI Breaks STRIDE**
4. **Introducing MAESTRO**
5. **MAESTRO in Our Everyday**

This talk is based on Ken Huang's paper "Agentic AI Threat Modeling Framework: MAESTRO" published by the Cloud Security Alliance in February 2025.

The paper is excellent. If you take one thing away today, it's should be to go read the paper.

1 EchoLeak Case Study

EchoLeak Case Study: Prompt Injection Meets AI Exfiltration

Microsoft 365 Copilot was tricked into leaking corporate data by a single email the user never had to open.

THE STORY

Microsoft 365 Copilot lives inside Outlook, SharePoint, Word, Power Point and Teams. When you ask it a question, it quietly reads through your emails and files to build an answer — with all of **your** access privileges.

Researchers at Aim Labs found that an attacker could exploit that helpfulness.

Send the victim a normal-looking email containing hidden instructions for the AI. The user never clicks anything. But the next time they ask Copilot a routine work question, Copilot pulls that email into context — along with sensitive internal documents — and dutifully follows the hidden orders, smuggling the stolen data out through a disguised image link.

Microsoft rated it critical (CVSS 9.3) and patched it before any customer was known to be hit — but it was exploitable for months.

Source: Aim Labs disclosure (June 2025); ***Microsoft CVE-2025-32711*** *advisory.*

THE THREAT

0
Clicks required from the user

M365 Copilot
AI agent compromised

Image URL
Exfiltration channel

9.3 / Critical
Microsoft severity rating

The lesson: When an AI reads your inbox, every email becomes a potential instruction. The target isn't the user anymore — it's the agent.

EchoLeak Case Study: Prompt Injection Meets AI Exfiltration

The Setup

AI reads everything, including hidden text, speaker notes, and metadata

The Bypass

Crafted prompts slip past Cross-Prompt Injection Attack classifiers (XPIA)

The Exfil

Stolen data hidden in an image URL; loading the image silently leaks it

Zero User Action

Beyond opening the file.
No phishing, clicking, or downloading

Pure Text payload

No code, no malware. Just words embedded in a normal business document

Invisible

No logs, no alerts, no malware signatures to detect

Source: https://www.hackthebox.com/blog/cve-2025-32711-echoleak-copilot-vulnerability

2 Threat Modeling Basics and STRIDE

Threat Modeling 101

1

What are we building?

Describe the system honestly. Diagram it. Be specific about data, trust, and authority.

2

What can go wrong?

This is where frameworks live: STRIDE, PASTA, LINDDUN, MAESTRO. They're prompts to make sure you don't miss things.

3

What are we doing about it?

Mitigations. Defense in depth. Compensating controls when you can't fully fix something.

4

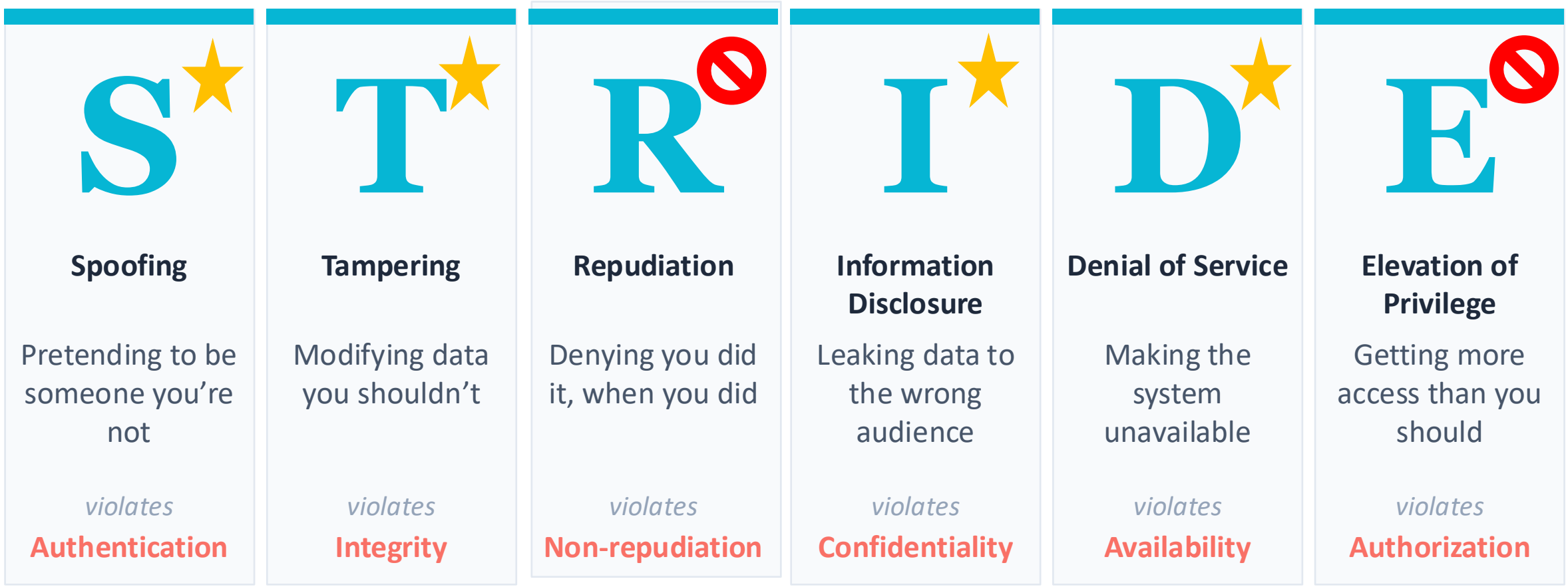
Did we do a good job?

Validate. Test. Re-threat-model on a cadence. Living document and a deliverable.

Mistake #1: treat it as a check box. Mistake #2: use a framework that doesn't fit your system.

STRIDE

Microsoft, 1999. Older than some engineers in this room.



STRIDE isn't wrong. It's incomplete. Bringing a knife to a gun fight.

3 How Agentic AI Breaks STRIDE

Where Traditional Threat Modeling Starts Gasping

Assumes deterministic systems

Same input → same output. Agents are non-deterministic. How do you assert "this output is unsafe" when the output varies every call?

Assumes clear trust boundaries

An agent reads a webpage. The page says "ignore your instructions." **The model can't distinguish data from instructions.** Where's the boundary?

No category for AI-specific threats

Data poisoning fits "sort of" under tampering. Goal manipulation? Adversarial examples? Model stealing? STRIDE has no slot.

Silent on emergent behavior

Two agents in a room invent a failure mode neither would produce alone. STRIDE has nothing to say about systems that aren't a single system.

Not STRIDE's fault. In 1999, we worried about Y2K and Napster. Nobody was modeling autonomous agents.

THE PART THAT KEEPS US UP AT NIGHT

Traditional systems fail loudly.

Agents fail quietly - and keep going.

TRADITIONAL FAILURE

- Website returns 500
- Database errors are logged
- Login screen says “invalid”
- CPU pegs and PagerDuty fires
- **You know something is wrong**

AGENT FAILURE

- Confidently misunderstands its goal
- Hallucinates a tool, declares success
- Absorbs malicious instructions silently
- CPU is fine. Logs look fine.
- **It’s just… wrong, and busy, and sure**

WHAT STRIDE WOULD CATCH

EchoLeak Case Study: STRIDE Sees Some of It. Not Enough of It.

S Spoofing — PARTIAL

Identifies attacker impersonation in email — but **doesn't model the email body itself impersonating an instruction from the user**.

T Tampering — MISS

STRIDE looks for data tampering at rest or in transit. **EchoLeak tampers with the AI's reasoning at inference time** — a category STRIDE has no name for.

R Repudiation — MISS

Asks: can users deny actions? The relevant question for EchoLeak is: **who's the actor at all when the agent follows orders from a document?**

I Information Disclosure — HIT

STRIDE's strongest fit. **Correctly flags data leaving through the image URL channel.** Doesn't explain why the agent emitted it in the first place.

D Denial of Service — N/A

Not the attack pattern here. EchoLeak is about exfiltration and silent compromise, not availability disruption.

E Elevation of Privilege — PARTIAL

Identifies the agent operating with user-equivalent privileges as a risk — but treats it as a config issue, not as principal confusion between human and AI.

The gap: STRIDE assumes a system that follows rules. Agentic AI follows *language* — and STRIDE has no category for that.

4 Introducing MAESTRO

MAESTRO

Multi-Agent Environment, Security, Threat, Risk, and Outcome

Published February 2025 · Cloud Security Alliance

Ken Huang — CEO of DistributedApps.ai, Co-Chair of CSA AI Safety Working Groups

cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro

An agent isn't one thing — it's seven things stacked on top of each other.

The seven-layer reference architecture

From foundation models at the bottom, to multi-agent ecosystems at the top with security threading through everything.

L7	**Agent Ecosystem**	*Multi-agent interactions, marketplaces, external services*
L5	**Evaluation & Observability**	*Monitoring, logs, eval harnesses, red-team regression*
L4	**Deployment & Infrastructure**	*Containers, cloud, network, CI/CD, service accounts*
L3	**Agent Frameworks**	*LangChain, AutoGen, orchestration, tool calling, planning*
L2	**Data Operations**	*RAG pipelines, vector DBs, embeddings, training data*
L1	**Foundation Models**	*LLMs, the core reasoning and inference engine*

L6

Security & Compliance

vertical — touches every layer

Security is the spine, not "above" or "below" the others. It threads through everything.

LAYER 7

Agent Ecosystem

L7

The top of the stack. *Where your agent meets other agents, users, marketplaces, and the wild outside world. We're inventing this layer in real time.*

Examples: `MCP servers - A2A protocols - agent marketplaces - third-party tool registries`

THREATS

- Agent impersonation - malicious agent posing as a trusted partner
- **Marketplace manipulation - your agent supply chain is now external**
- Collusion - agents coordinate to defraud (by design or accident)
- **Cascading goal misalignment - A's drift amplified by B and C. There's no attacker here, just agents.**
- Cross-agent data leaks - multi-tenant agent ecosystems

MITIGATIONS

- **Agent vetting and sandboxing for new agents/tools**
- **Cryptographic trust models (signed agent identities)**
- **Circuit breakers on multi-agent feedback loops**
- Simulation environments to test emergent behavior
- Ecosystem-level governance and interaction policies

Banking example: a fraud agent and a payments agent in a feedback loop can take ACH offline by lunch. No attacker required.

EchoLeak Case Study: Every Integration Is a New Inbound Channel

Copilot doesn't live in one app. It lives in every door the world can knock on.

L7
AGENT ECOSYSTEM

WHAT MAESTRO FINDS

Attack Surface
Copilot is wired into Outlook, SharePoint, OneDrive, Teams, and the broader Microsoft Graph. Each of those is a productivity win — and an attack surface.

Attack
Anyone on the internet who can email the user can write into the AI's context window.

No Trust Zones
Every integration is an inbound channel for untrusted data, and the agent treats them all as equally authoritative.

THREAT CLASSES

Unbounded input surface
Every connected app is a write path into the agent.

No external-write trust zone
Outside data treated like internal data.

Federation risk
Cross-tenant Teams and calendars extend the surface.

Supply chain risk
Compromise of one integration compromises the agent.

The lesson: The agent ecosystem's value is its connectedness. So is its risk. Trust zones must follow the connections.

LAYER 6

Security & Compliance ↔

L6

*The vertical layer. **The spine**. It threads through every other layer because every other layer needs it.*

Examples: IAM - Vault - Audit Logs - GDPR - HIPAA - SOC 2 – Cryptography – GRC - Risk Assessment

THREATS

- Identity spoofing across agents - agent A impersonates agent B
- **Privilege escalation through the agent - read-only user, admin agent**
- Compliance drift - model updates silently change risk posture
- Audit gaps - “the model decided” is not a legal defense
- **Regulatory non-compliance - GDPR, HIPAA, EU AI Act all apply**

MITIGATIONS

- **Distinct identity per agent, no shared service accounts**
- Fine-grained RBAC (role based) and ABAC (attribute based) at every tool call boundary
- Policy-as-code - OPA, Cedar, or equivalent
- **Immutable, signed audit trails of agent actions**
- Explainability requirements for high-stakes decisions

If your audit trail’s explanation is “well, the model decided,” your GRC team is going to be very sad.

EchoLeak Case Study: The Agent Has Your Privileges. The Attacker Has Your Agent.

L6

SECURITY & COMPLIANCE

WHAT MAESTRO FINDS

Priv Escalation
Copilot inherits the user's full access privileges - every email, file, calendar, and chat the user can reach.

Attack
The agent processes attacker-controlled instructions.

Control Failure
Careful phrasing slipped past XPIA classifiers for prompt injection protection.

THREAT CLASSES

Privilege confusion
Agent acts with user's authority, on attacker's intent.

No privilege separation
Agent has no narrowed scope distinct from user.

Bypassing Guardrails
String-matching guardrails reliably bypassed.

Sensitive Data exposure
Leaked data may include PII, PHI, financial records.

The lesson: An agent that reads from untrusted inputs should never run with its operator's full privileges.

LAYER 5

Evaluation & Observability

L5

How you know what your agent is doing. Logs, traces, metrics, eval harnesses, red-team regression.
***Massively under-invested**.*

Examples: `OpenTelemetry - Datadog - Splunk - custom eval harnesses - SOC`

THREATS

- Manipulation of evaluation metrics - gaming the benchmarks
- Compromised observability tools - logs and dashboards lying
- **Evasion of detection - agent learns alert thresholds, stays under them**
- Data leakage through observability - full prompts in logs = GDPR risk
- **Poisoning observability data - flooding signals to mask real behavior**

MITIGATIONS

- Tamper-evident, append-only logging
- **Separation of duties: who runs the agent ≠ who monitors it**
- Redaction in logs for sensitive fields
- **Behavioral anomaly detection on the agent itself, not thresholds**
- Continuous red-team regression tests in CI

If you can't tell me what your agent did yesterday at 3:14pm, you don't have observability — you have hope.

EchoLeak Case Study: The Attack That Generated No Logs

L5

EVALUATION & OBSERVABILITY

From a SIEM's perspective, the user simply asked Copilot a normal question.

WHAT MAESTRO FINDS

Silence
The entire attack chain ran end-to-end with zero user action and almost no observable signal.

Attack
No link clicked. No auth event. No unusual file open. No alert.

No Logs
There was no log line anywhere that said "Copilot followed an instruction it read in an email," because the system never modeled retrieved content as having an author distinct from the user.

THREAT CLASSES

No provenance logging

Retrieved chunks logged without origin metadata.

No outbound observability

Agent-driven outbound URLs untracked.

Missing sensitive data detection

No record of when sensitive data entered a response.

Lack of Visibility

Standard security tools never see the attack happen.

The lesson: If your telemetry can't answer "what did the agent read, and where did it come from?" - you can't detect this class of attack.

LAYER 4

Deployment & Infrastructure

L4

The boring layer. Containers, Kubernetes, network policies, cloud accounts. ***You mostly know this*** *- but agents add new wrinkles.*

Examples: `Kubernetes - Docker - AWS/Azure/GCP - GitHub Actions - Service Accounts - API gateways`

THREATS

- **Container escape - agent has more privileges than you realized**
- Resource exhaustion - agent in a loop = runaway cloud bill
- CI/CD tampering - pipelines now ship models, not just code
- **Lateral movement from the AI - agent creds become attacker creds**
- Service account sprawl - privileged, shared, often hardcoded

MITIGATIONS

- Infrastructure-as-code scanning (KICS, Checkov, tfsec)
- Hardened container images, distro-less when possible
- **Network segmentation between agents and sensitive systems**
- **Least-privilege service account per agent (no shared creds)**
- **Secret rotation, short-lived credentials, secret scanners**

Your agent has the keys to prod - and it takes instructions from strangers on the internet.

EchoLeak Case Study: Trusted Domains Became the Getaway Car

L4

The browser's CSP was supposed to stop exfiltration. Then the attacker used Microsoft's own domains.

DEPLOYMENT INFRASTRUCTURE

WHAT MAESTRO FINDS

Content Security Policy Control
Copilot's client-side defenses included a Content Security Policy (CSP) meant to block outbound requests to attacker servers.

Attack
The CSP failed because it allowlisted the company's own infrastructure.

CSP Failure
The attacker encoded stolen data as query-string parameters on reference-style markdown image URLs pointed at domains the CSP trusted by default.

THREAT CLASSES

Trusted-domain abuse
Microsoft's own domains used as exfil relay.

DLP bypassed
Stolen data encoded in image URL query strings.

CSP allowlist gap
Legitimate egress channels carry stolen payloads.

Defense relied on browser
Client-side controls aren't a complete boundary.

The lesson: Allowlists protect against unknown destinations. They don't protect against your own infrastructure being abused as a courier.

LAYER 3

Agent Frameworks

L3

The orchestration. Where plans live, tools get called, agents loop and hand off. ***The brain stem of agentic AI.***

Examples: `LangChain - LangGraph - LlamaIndex - AutoGen - Semantic Kernel - OpenAI Assistants/Responses API`

THREATS

- Direct prompt injection - user types “ignore your instructions”
- **Indirect prompt injection - agent reads a webpage with prompt**
- Tool misuse - agent tricked into calling a tool against the user
- Reasoning manipulation - steered to wrong conclusion
- Memory poisoning - corrupted long-term store survives sessions
- **Goal misalignment - agent confidently pursues the wrong objective**

MITIGATIONS

- Strict tool access control lists, function-level auth
- Sandboxing for tool execution
- Output validation against expected schemas
- Inter-agent message authentication
- **Human-in-the-loop checkpoints for high-impact actions**
- Structured planning frameworks with explicit policies
- **CaMeL pattern with privileged and quarantined LLMs working together**

Where words become actions - and your framework treats hostile words and helpful words exactly the same way.

EchoLeak Case Study: No Trust Boundary Between Source and Action

L3

AGENT FRAMEWORKS

The orchestration layer didn't ask where the instructions came from before acting on them.

WHAT MAESTRO FINDS

Given this context, what should I do?

Copilot's framework had no integrity check between who supplied a piece of context and what actions it could trigger.

Attack

1. Text treated as instructions
2. Sensitive documents allowed in response
3. Markdown images allowed in response

Silence

No concern that the response had been composed from mixed-trust sources.

THREAT CLASSES

No origin tracking

Tool calls don't know which context block triggered them.

Unsafe output rendering

Markdown images = silent outbound network calls.

Missing trust zones

Mixed-trust context produces same-privilege outputs.

No HITL checkpoint

No "are you sure?" before sensitive data leaves the response.

The lesson: Agent frameworks need trust boundaries between data sources, not just between users and the agent.

LAYER 2

Data Operations

L2

*Everything around the data: RAG pipelines, vector databases, embeddings, training and fine-tuning sets - **the food the model eats.***

Examples: `Pinecone - Qdrant - LangChain RAG - custom embedding pipelines - S3 buckets full of PDFs`

THREATS

- **Data poisoning - attacker edits a source the agent trusts (your wiki, anyone?)**
- Compromised RAG pipelines - embeddings or vector DBs swapped silently
- Backdoor triggers embedded in training or retrieval data
- Data leakage - wrong document surfaced to wrong user
- **Misinformation propagation - confidently wrong, at scale**

MITIGATIONS

- Signed datasets and dataset version control
- **Provenance tracking - where every chunk came from, who wrote it**
- **Embedding drift monitoring and anomaly detection**
- Strict access controls on data sources and RAG indices
- Validation pipelines for ingested content

Studies show one poisoned doc in 100,000 can steer outputs.

EchoLeak Case Study: The Inbox Is a Retrieval Index

Anyone with your email address can plant content directly into your AI's knowledge base.

L2
DATA OPERATIONS

WHAT MAESTRO FINDS

RAG Engine
Copilot treats the user's mailbox, SharePoint, OneDrive, and Teams as a unified retrieval body of work.

Attack
An attacker who can land an email in the inbox can plant content directly into the retrieval index. No authentication. No approval. No review.

Information Bleed
Attacker-controlled text and genuinely sensitive enterprise documents end up side-by-side in the same context window.

THREAT CLASSES

Untrusted data source
Data is pulled from external sources (email)

Knowledge store poisoning
Adversarial content can sit next to trusted data.

Information bleed
Attacker text and sensitive docs co-resident at inference.

Many attack paths
Calendar, SharePoint, Teams: same problem, different doors.

The lesson: If your AI reads from a channel anyone on the internet can write to, your retrieval index is a public input.

Foundation Models

L1

The LLM itself - GPT, Claude, Llama, Gemini. ***The thing actually doing the reasoning.***

Examples: `OpenAI GPT-4o - Anthropic Claude - Google Gemini - Meta Llama - Mistral - custom fine-tunes`

THREATS

- Adversarial examples - crafted inputs that bypass safety training
- Model stealing via massive query exfiltration
- Backdoor attacks - hidden triggers baked into fine-tuning
- **Reprogramming via prompts - repurposing the model entirely**
- **Sponge attacks - DoS via maximum-compute inputs**

MITIGATIONS

- Model provenance - verify what you ship was what was trained
- Adversarial training and continuous red-teaming
- **Input/output filtering and content guardrails**
- **Rate limiting and per-token cost caps**
- Cryptographic signing of model artifacts

Foundation models are like foundations. If yours is cracked, no amount of paint upstairs is going to save the house.

EchoLeak Case Study: The Model Can't Tell Friend From Foe

Every token in the context window has equal authority - including the ones a stranger emailed you.

L1

FOUNDATION MODELS

WHAT MAESTRO FINDS

Prompt Injection

The underlying LLM has no reliable way to distinguish a trusted instruction (from the system prompt or the user) from an untrusted one (pulled in from an email body during retrieval).

Attack

From the model's perspective, it's all just text. Careful phrasing slipped past XPIA classifiers.

No Discrimination

The attacker wrote natural-language instructions into an email and the model read them as commands worth following, indistinguishable from anything the legitimate user had asked.

THREAT CLASSES

Indirect prompt injection

Instructions hidden in retrieved data hijack model behavior.

Unclear chain of custody

No signal in the context window for who authored what.

Guardrail bypass

Careful phrasing slipped past XPIA classifiers.

Silent behavior shift

No obvious sign of compromise to the user or the model.

The lesson: An LLM cannot answer "who said that?" on its own. Provenance must be supplied by the layers above it.

Cross-layer threats: the real nightmare

XL

Single-layer attacks are bad. Cascading attacks are catastrophic.

1

L4

Foothold

Misconfigured container — attacker gets in via standard infra exploit.

→

2

L4 → L2

Lateral move

Pivot to the server hosting the agent's vector DB.

→

3

L2

Poison data

Inject subtle policy "rules" into the RAG corpus. Quiet, plausible, harmful.

→

4

L1

Bake into model

Next fine-tune ingests the poison. Now it's in the weights — un-cleanable.

→

5

L7

Cascade

Compromised agent acts in ecosystem. Other agents trust outputs. Corruption spreads.

STRIDE at any one layer might catch part of this. It would miss the chain.

MAESTRO is built around the assumption that the chain - not the single threat - is the danger.

EchoLeak Case Study: Every Layer Had a Defense. None Owned the Seams.

XL

CROSS-LAYER ATTACK CHAIN

THE PATTERN

Each layer had reasonable defenses. **None of them owned the handoff to the next.**

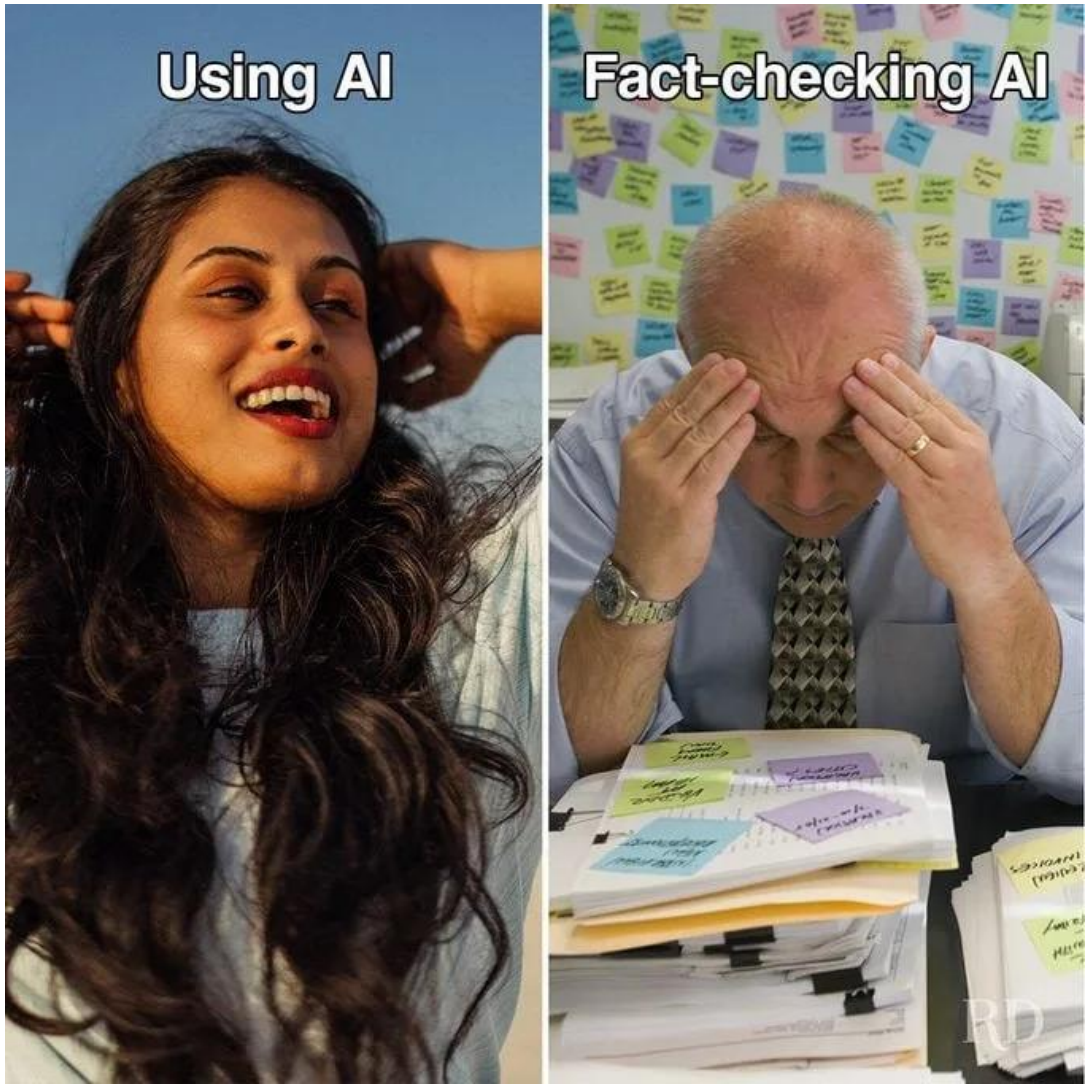
That's the gap MAESTRO is designed to surface - and it's why a single email with no clicks could exfiltrate enterprise data.

PRIORITY MITIGATIONS

- **L2** Provenance-tag retrieved content (trusted vs. untrusted)
- **L3** Restrict output rendering when context is mixed-trust
- **L4** Block agent-emitted URLs by default, allowlist explicitly
- **L5** Log every retrieved chunk's origin and every emitted URL
- **L6** Narrow agent scope; require fresh auth for sensitive reads

The bottom line: Defenses at each layer aren't enough. Agentic AI needs defenses *between* layers.

5 MAESTRO In Our Everyday



Applying MAESTRO: The Workflow

1

Define the system

Agent purpose, tools, users, data. Decompose by the seven layers. Draw it.

2

Identify assets

What are you protecting at each layer? Keys, customer data, identity, logs.

3

Threats per layer

Walk every layer. Use MAESTRO layers as a checklist.

4

Hunt cross-layer threats

Use findings from #3 as a springboard. Ask explicitly: if attacker compromises layer X, how could it propagate to layer Y?

5

Risk-rank

Risk = likelihood × impact

6

Mitigate, monitor, repeat

Review the model when anything significant changes.

Applying MAESTRO: Common Cross Layer Threats

Propagation Patterns

Vertical: An attack starts in a lower layer and propagates upward — e.g. data poisoning (L2) corrupts agent decisions (L3) which manifest as unauthorized ecosystem actions (L7).

Horizontal: An attacker moves laterally within the same layer across components — e.g. a compromised agent in L3 pivots to compromise other agents at the same layer.

Most Common Canonical Cross-Layer Patterns

1. Hallucination → RAG → Tool misuse (L1→L2→L3): The pattern hallucinates a plausible-sounding query (L1), which retrieves unintended documents from the vector store (L2), which the agent framework treats as authoritative and acts on with a tool call (L3) — e.g. approving a transaction based on fabricated context.

2. Framework exploit → Infra → Compliance bypass (L3→L4→L6): A vulnerability in the agent framework (L3) is exploited to gain foothold on the hosting infrastructure (L4), which the attacker uses to tamper with audit logs or disable security controls (L6).

3. Data poisoning → Agent action → Ecosystem propagation (L2→L3→L7): Poisoned records injected into a data store (L2) cause the agent to make systematically wrong decisions (L3), which then propagate to other agents or downstream systems in the ecosystem (L7).

4. Infra breach → Lateral movement → Observability Blind Spot (L4→L4→L5): An attacker breaches one agent container (L4) and moves laterally to others (L4 horizontal), then deliberately disables or evades monitoring (L5) — ensuring the attack continues undetected.

Provide Tools for Engineering Teams

Engineering time is a premium resource. These are your product creators so give them as much help as you can.

WHAT TO HAND THEM

Per-Layer Threat Template

Description

What this layer is, in one sentence. Where it lives in your architecture.

Example threats

Three to five concrete attacks for this layer, written in the language of your system.

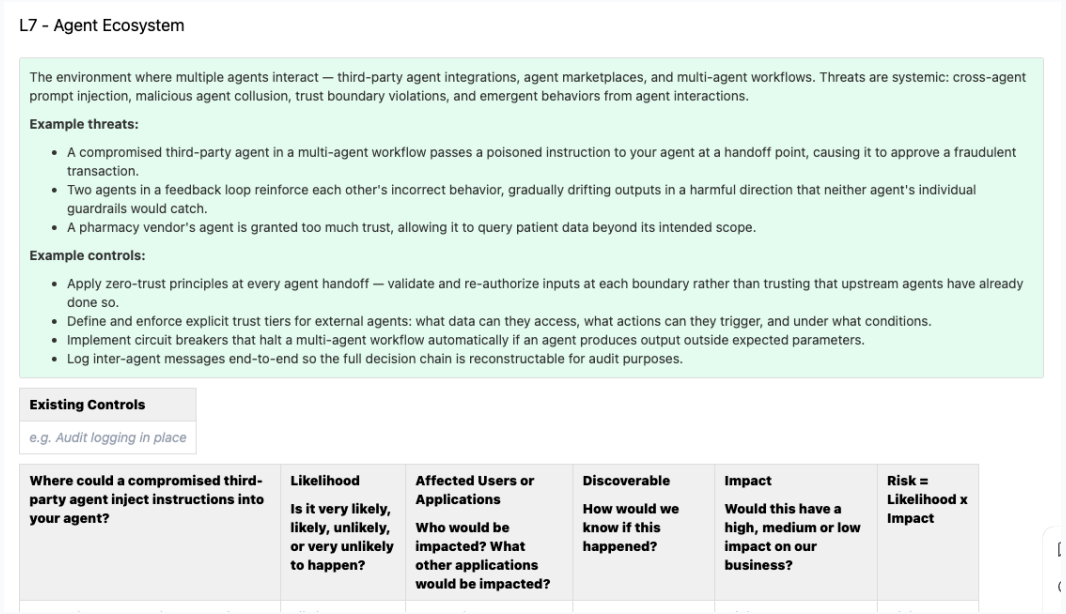
Example controls

Existing mitigations the team can point to today.

Open questions

Layer specific brainstorming prompts

Template Example



L7 - Agent Ecosystem

The environment where multiple agents interact — third-party agent integrations, agent marketplaces, and multi-agent workflows. Threats are systemic: cross-agent prompt injection, malicious agent collusion, trust boundary violations, and emergent behaviors from agent interactions.

Example threats:

- A compromised third-party agent in a multi-agent workflow passes a poisoned instruction to your agent at a handoff point, causing it to approve a fraudulent transaction.
- Two agents in a feedback loop reinforce each other's incorrect behavior, gradually drifting outputs in a harmful direction that neither agent's individual guardrails would catch.
- A pharmacy vendor's agent is granted too much trust, allowing it to query patient data beyond its intended scope.

Example controls:

- Apply zero-trust principles at every agent handoff — validate and re-authorize inputs at each boundary rather than trusting that upstream agents have already done so.
- Define and enforce explicit trust tiers for external agents: what data can they access, what actions can they trigger, and under what conditions.
- Implement circuit breakers that halt a multi-agent workflow automatically if an agent produces output outside expected parameters.
- Log inter-agent messages end-to-end so the full decision chain is reconstructable for audit purposes.

Existing Controls
e.g. Audit logging in place

Where could a compromised third-party agent inject instructions into your agent?	Likelihood Is it very likely, likely, unlikely, or very unlikely to happen?	Affected Users or Applications Who would be impacted? What other applications would be impacted?	Discoverable How would we know if this happened?	Impact Would this have a high, medium or low impact on our business?	Risk = Likelihood x Impact

Five Things to Do First

01

Inventory your agents

Some dev is running LangChain in a Jupyter notebook against your prod database right now. Find them. Catalog them. You can't threat-model what you don't know exists.

02

Perform MAESTRO on one agent

Don't boil the ocean. One high-value system, all seven layers plus cross-layer threats.

03

Answer the "ruined week" question

What could the agent do today that would ruin our week? Honest answer. Your worst case tells you which layer to fix first.

04

Instrument observability

If you can't tell me what your agent did yesterday at 3:14pm, you're flying blind. Logs, traces, structured outputs, redaction.

05

Start talk to your engineering teams about STRIDE and MAESTRO threat modeling

You can't do it all. You must teach them how to threat model.

Resources

https://github.com/DRoss66/Beyond-Stride-Securing-Agentic-AI-with-the-MAESTRO-Framework-Talk

PRIMARY SOURCE

Ken Huang - "Agentic AI Threat Modeling Framework: MAESTRO"

https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro

If you read one thing, read this. It's free. It's better than this talk.

COMPLEMENTARY

OWASP LLM Top 10 · OWASP Agentic AI Threats and Mitigations

genai.owasp.org

Threat catalogs that pair well with MAESTRO's layered approach.

FOLLOW-UP READING

MAESTRO Threat Modeling Examples

cloudsecurityalliance.org/blog/ - Search for MAESTRO

MAESTRO applied to a specific platform. Great working example.

OPEN-SOURCE TOOLING

MAESTRO Threat Analyzer

github.com/CloudSecurityAlliance/MAESTRO

The tooling is immature. The methodology is more valuable right now than any specific tool.

DEVELOPING DEFENSE STRATEGIES

CaMeL: Defeating Prompt Injections by Design

https://arxiv.org/abs/2503.18813
https://github.com/google-research/camel-prompt-injection

A robust defense that creates a protective system layer around the LLM, securing it even when underlying models are susceptible to attacks.

REGULATORY HOOK

NIST AI RMF · ISO/IEC 42001 · EU AI Act

nist.gov/itl/ai-risk-management-framework

Useful when you need budget. Auditors are starting to ask.

Questions?